MINI-COURIER PLANNER

Integrated Algorithm Design and Performance Evaluation

Capstone Project Report

Student Name: _____

Roll Number: _____

Course / Section: _____

University: _____

Fill in the student details above and verify the results on your own computer before submission.

1. Abstract

The Mini-Courier Planner is a Python prototype integrating sorting, dynamic programming, graph algorithms, and approximation techniques. It prioritizes parcels, selects a feasible set under a vehicle weight limit, calculates shortest paths in a weighted delivery network, estimates a complete delivery tour, and records execution times.

2. Problem Statement and Objectives

Given parcels and delivery locations, develop an ordered delivery and packing plan. The application should prioritize parcels, maximize selected parcel priority subject to vehicle capacity, calculate shortest paths, approximate a complete route, and compare algorithm runtimes.

• Sort parcels by delivery deadline, then higher priority.

• Select parcels using 0/1 knapsack dynamic programming.

• Represent delivery points as a weighted graph.

• Use Dijkstra's algorithm for shortest paths.

• Use nearest-neighbor TSP approximation for a closed tour.

• Benchmark major components and discuss complexity.

3. Dataset and Assumptions

The included datasets are synthetic demonstration data. Parcel fields are ID, customer, deadline in hours, priority score, and weight in kg. The graph has five locations including depot D; edge weights are illustrative distances in km. The graph is undirected with non-negative weights.

4. System Design and Methodology

Sorting: Python's stable sort orders parcels by deadline ascending, then priority descending, then ID.

Knapsack: A 0/1 dynamic-programming table stores the maximum priority value for each parcel prefix and capacity. Backtracking reconstructs the selected set.

Dijkstra: An adjacency-list graph and min-heap compute shortest distances from a source. Non-negative edge weights are required.

Nearest neighbor: Starting at the depot, repeatedly select the unvisited location with the smallest shortest-path distance from the current location, then return to the depot. This heuristic is not guaranteed to find an optimal TSP tour.

5. Algorithm Complexity

Component	Time complexity	Explanation
Sorting	O(n log n)	Stable comparison sort
0/1 Knapsack	O(nW)	n parcels, capacity W; table space O(nW)
Dijkstra	O((V+E) log V)	Binary heap and adjacency list
Nearest neighbor	Repeated Dijkstra runs	Approximation; shortest-path calls dominate

6. Sample Delivery Network

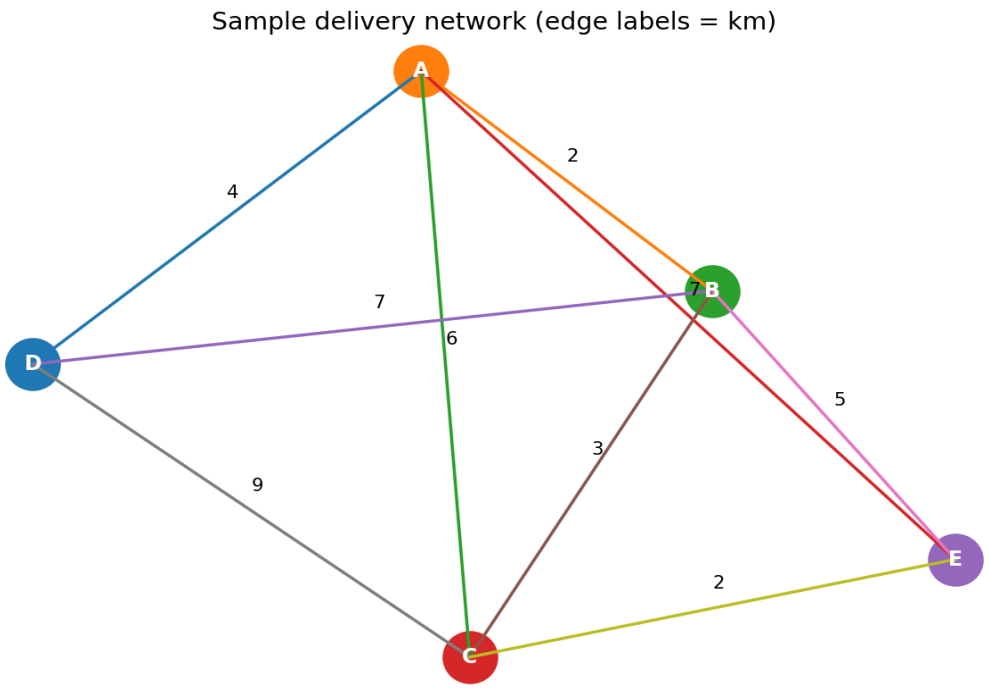


Figure 1. Sample weighted delivery network; edge labels are illustrative kilometres.

7. Runtime Experiment and Results

The benchmark tests input sizes 20, 50, 100, and 200. Sorting and knapsack use generated parcel lists; knapsack capacity is fixed at 60 kg. Dijkstra runs on a sparse connected graph with a chain and periodic shortcut edges. A fixed random seed makes generated test data reproducible; elapsed times still vary by machine and system load.

Input size	Sorting (ms)	Knapsack (ms)	Dijkstra (ms)
20	0.0062	0.1272	0.0161
50	0.0103	0.3250	0.0309
100	0.0200	0.6143	0.0679
200	0.0463	1.2106	0.1198

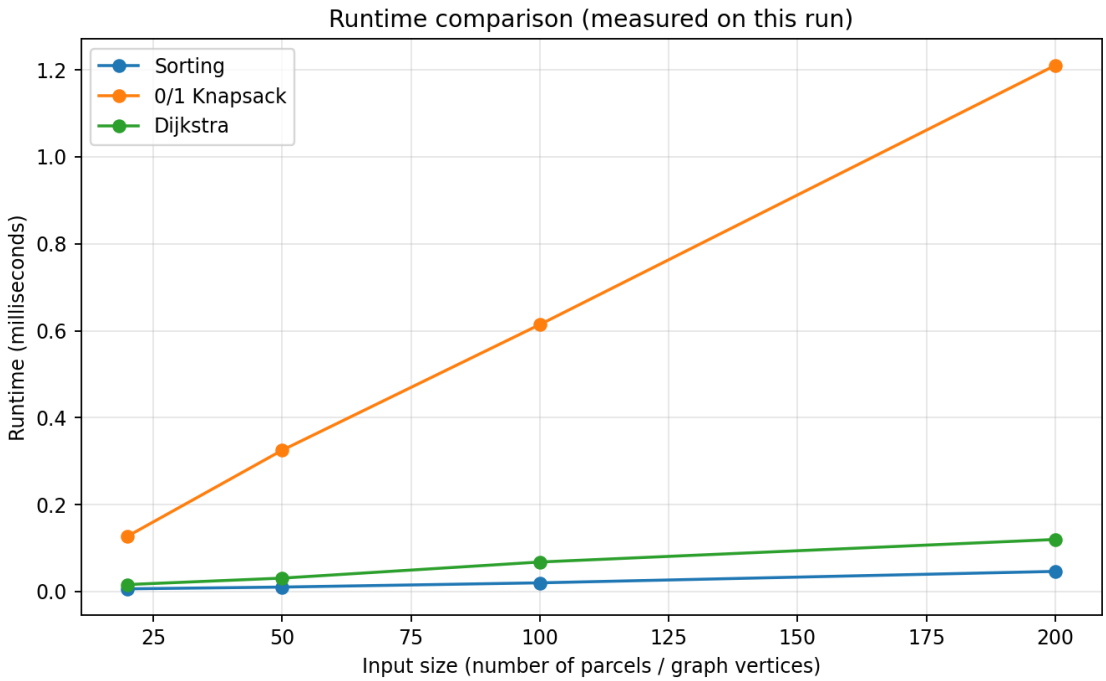


Figure 2. Runtime comparison measured during this run. Treat the values as machine-specific observations.

8. Testing and Validation

• Sorted output follows the deadline/priority key.

• The selected parcel weight does not exceed the configured 18 kg vehicle capacity.

• Dijkstra computes distances from depot D over non-negative edges.

• The route visits each graph node and returns to the depot.

• Benchmark values are saved in results/benchmark_results.json.

Console output captured from main.py

```
============================================================
                MINI-COURIER PLANNER
============================================================

1) PARCEL PRIORITIZATION (deadline, priority)
  P05  | deadline= 3h | priority=10 | weight= 8kg
  P01  | deadline= 5h | priority=10 | weight= 6kg
  P03  | deadline= 8h | priority= 9 | weight= 7kg
  P07  | deadline=10h | priority= 8 | weight= 6kg
  P02  | deadline=12h | priority= 7 | weight= 5kg
  P06  | deadline=18h | priority= 6 | weight= 3kg
  P04  | deadline=24h | priority= 5 | weight= 4kg
  P08  | deadline=30h | priority= 4 | weight= 2kg

2) PACKING PLAN (vehicle capacity = 18 kg)
  SELECT P01  weight=6kg  priority=10
  SELECT P03  weight=7kg  priority=9
  SELECT P06  weight=3kg  priority=6
  SELECT P08  weight=2kg  priority=4
  Total weight: 18 kg
  Total priority value: 29

3) SHORTEST PATHS FROM DEPOT
  D -> D: 0.0 km
  D -> A: 4.0 km
  D -> B: 6.0 km
  D -> C: 9.0 km
  D -> E: 11.0 km

4) APPROXIMATE DELIVERY TOUR (nearest-neighbor)
  D -> A -> B -> C -> E -> D
  Tour distance: 22.0 km

5) SAMPLE SHORTEST PATH RECONSTRUCTION
  D -> A: D -> A
```

Figure 3. Console output captured from executing the application.

9. Limitations

This is an educational prototype, not a live dispatch system. It does not use GPS, traffic, delivery time windows, multiple vehicles, or real addresses. Knapsack maximizes priority score, not deadline satisfaction. Nearest-neighbor is a heuristic and may not give the shortest possible tour. Runtime measurements depend on the execution environment.

10. Conclusion and Future Scope

The prototype demonstrates integration of sorting, dynamic programming, shortest paths, and an approximation heuristic. Future extensions may include multiple vehicles, delivery time windows, live road-network data, a graphical interface, route-improvement heuristics, and automated unit tests.

11. Execution Instructions

Install Python 3.9 or newer. From the project root, run `pip install -r requirements.txt`, then `python main.py`. To open the notebook, run `jupyter notebook mini_courier_planner.ipynb`.

12. Final Submission Checklist

■ README.md and requirements.txt

■ mini_courier_planner.ipynb and main.py

■ src/ algorithm modules

■ datasets/ parcel and route files

■ images/ route graph, runtime chart, console output

■ results/ benchmark data and console log

■ Project_Report.pdf with student details completed

■ Private GitHub repository with final tag v1.0-final-submission